Arrays

Mahir Labib Dihan
Lecturer, CSE, BUET

Table of Contents

Introduction to Arrays

The Problem: Storing Multiple Values

Scenario: You need to store marks of 50 students.

Without Arrays:

- Declare 50 separate variables: int mark1, mark2, mark3, ..., mark50;
- Write 50 `scanf()` statements to read input
- Write 50 statements to calculate average
- What if you need 1000 students? 10000?

With Arrays:

- `int marks[50];` – One line declares 50 integer variables!
- Use loops to read, process, and output data
- Scale to any size easily

Arrays solve the problem of managing collections of similar data.

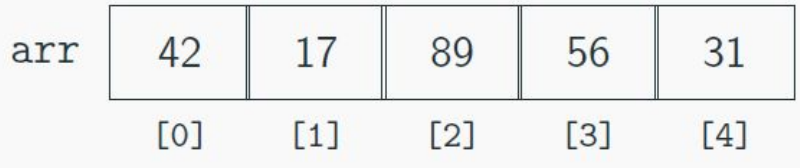
What is an Array?

Definition

An **array** is a collection of elements of the same data type, stored in contiguous (adjacent) memory locations, accessed using an index.

Key Properties:

- **Homogeneous:** All elements must be of the same type
- **Fixed Size:** Size is determined at declaration (in standard C)
- **Zero-Indexed:** First element is at index 0, not 1
- **Contiguous:** Elements are stored next to each other in memory



Array Declaration

Syntax

```
data_type array_name[size];
```

Examples:

```
int marks[50]; // 50 integers
float temperatures[7]; // 7 floating - point numbers
char name[20]; // 20 characters (a string)
double prices[100]; // 100 double - precision numbers
```

Important Notes:

- Size must be a positive integer constant (or constant expression)
- Arrays declared inside functions have garbage values initially
- Global arrays are initialized to zero by default

Array Initialization

Method 1: Initialize at Declaration

```
int arr[5] = {10 , 20, 30, 40, 50}; // All 5 values specified
```

Method 2: Partial Initialization

```
int arr[5] = {10 , 20, 30, 40, 50}; // All 5 values specified
```

Method 3: Initialize All to Zero

```
int arr[5] = {0}; // All elements are 0
```

Method 4: Let Compiler Determine Size

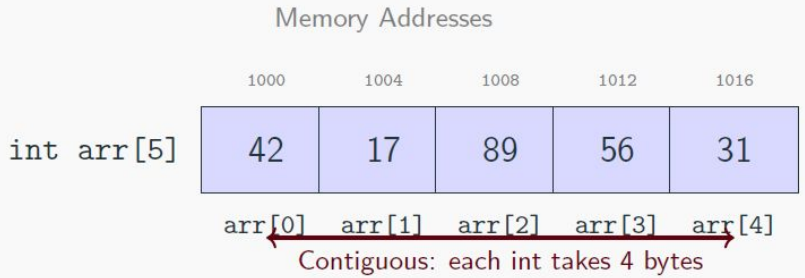
```
int arr[] = {1, 2, 3, 4, 5}; // Size automatically set to 5
```

Method 5: Initialize Using Loop

```
int arr[5];
for (int i = 0; i < 5; i++) {
     arr[i] = i * 10; // arr = {0, 10, 20, 30, 40}
}
```

Array Memory Layout

How arrays are stored in memory:



Key Insight: If `arr` starts at address `1000` and each `int` is 4 bytes:

- `arr[0]` is at address `1000`
- `arr[1]` is at address `1004`
- `arr[i]` is at address `1000 + i × 4`

Working with Arrays

Accessing Array Elements

Syntax: `array name[index]`

```
int arr[5] = {10 , 20, 30, 40, 50};
// Reading values
int first = arr[0]; // first = 10
int third = arr[2]; // third = 30
// Modifying values
arr[1] = 25; // arr = {10 , 25, 30, 40, 50}
arr[4] = arr[0] + arr[2]; // arr[4] = 10 + 30 = 40
// Using variables as index
int i = 3;
printf("%d\n", arr[i]); // Prints 40
```

Warning: Array Bounds

C does not check array bounds! Accessing `arr[5]` or `arr[-1]` causes undefined behavior (crash, garbage, or silent corruption).

Assigning Values to Arrays

```c
#include <stdio.h>

int main() {
	int arr[5];
	// Method 1: Assign individually
	arr[0] = 10;
	arr[1] = 20;
	arr[2] = 30;
	arr[3] = 40;
	arr[4] = 50;
	// Method 2: Assign using a loop (preferred for patterns)
	for (int i = 0; i < 5; i++) {
		arr[i] = (i + 1) * 10; // arr = {10 , 20, 30, 40, 50}
	}
	// Method 3: Assign based on some formula
	for (int i = 0; i < 5; i++) {
		arr[i] = i * i; // arr = {0, 1, 4, 9, 16}
	}
	return 0;
}
```

Reading Array Input from User

```c
#include <stdio.h>

int main () {
	int n;
	printf("Enter number of elements: ");
	scanf("%d", &n);
	int arr[n]; // Variable length array
	printf ("Enter %d elements:\n", n);
	for (int i = 0; i < n; i++) {
		scanf("%d", &arr[i]); // Note: &arr[i] for address
	}
	// Print the array
	printf("You entered : ");
	for (int i = 0; i < n; i++) {
		printf("%d ", arr[i]);
	}
	printf("\n");

	return 0;
}
```

Finding Array Size with sizeof

```c
#include <stdio.h>
int main() {
      int arr[] = {10 , 20, 30, 40, 50};
      // sizeof(arr) gives total bytes
      // sizeof(arr[0]) gives bytes per element
      int size = sizeof(arr) / sizeof(arr[0]);
      printf("Array size: %d\n", size); // Output: 5
      // Use in loops
      for (int i = 0; i < sizeof(arr)/ sizeof(arr[0]); i++) {
            printf("%d ", arr[i]);
      }
      return 0;
}
```

Warning

This only works for arrays in the same scope. When passed to functions, arrays decay to pointers and `sizeof` won't work correctly!

Passing Arrays to Functions

Arrays are **always passed by reference** (as pointers) – no copy is made.

```
void printArray(int arr[], int size) {
      for (int i = 0; i < size ; i++) {
            printf("%d ", arr[i]);
      }
      printf("\n");
}
void doubleElements(int arr[], int size) {
      for (int i = 0; i < size ; i++) {
            arr[i] *= 2; // Modifies original array!
      }
}
int main () {
      int nums[] = {1, 2, 3, 4, 5};
      printArray(nums , 5); // 1 2 3 4 5
      doubleElements(nums , 5);
      printArray(nums , 5); // 2 4 6 8 10 (modified!)
      return 0;
}
```

Some Array Problems

Problem: Sum and Average

Task: Calculate sum and average of array elements.

```c
#include <stdio.h>

int main () {
     int arr[] = {10 , 20, 30, 40, 50};
     int n = sizeof(arr) / sizeof(arr[0]) ;

     int sum = 0;
     for (int i = 0; i < n; i++) {
          sum += arr[i];
     }

     float average = (float) sum / n;

     printf("Sum = %d\n", sum); // Sum = 150
     printf("Average = %.2f\n", average); // Average = 30.00

     return 0;
}
```

Problem: Find Maximum and Minimum

Task: Find the largest and smallest elements.

```c
#include <stdio.h>

int main () {
     int arr[] = {34 , 12, 89, 45, 23, 67};
     int n = sizeof(arr) / sizeof(arr[0]);

     int max = arr[0]; // Assume first element is max
     int min = arr[0]; // Assume first element is min

     for (int i = 1; i < n; i++) {
          if (arr[i] > max) max = arr[i];
          if (arr[i] < min) min = arr[i];
     }

     printf("Maximum = %d\n", max); // 89
     printf("Minimum = %d\n", min); // 12
     return 0;
}
```

Problem: Find Maximum and Minimum

Task: Find both the largest and second largest elements in a single pass.

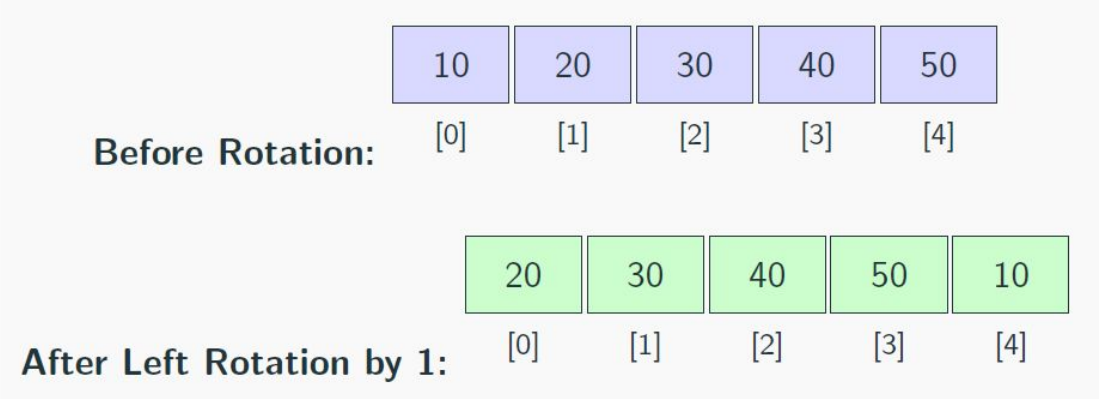
Approach:

- Track two variables: first (largest) and second
- When we find a new largest, old largest becomes second
- When we find something between first and second, update second

```
int arr[] = {12 , 35, 1, 10, 34, 1};
int n = 6;
int first = arr[0], second = -1; // Use INT_MIN for general case
for (int i = 1; i < n; i++) {
     if (arr[i] > first ) {
          second = first; // Old first becomes second
          first = arr[i];
     } else if (arr[i] > second && arr[i] != first ) {
          second = arr[i];
     }
}
printf("First: %d, Second: %d\n", first, second); // 35, 34
```

Left Rotate Array: Concept

Task: Rotate all elements to the left by one position.



1. Save the first element
2. Shift all elements one position to the left
3. Put the saved element at the last position

Left Rotate Array: Solution

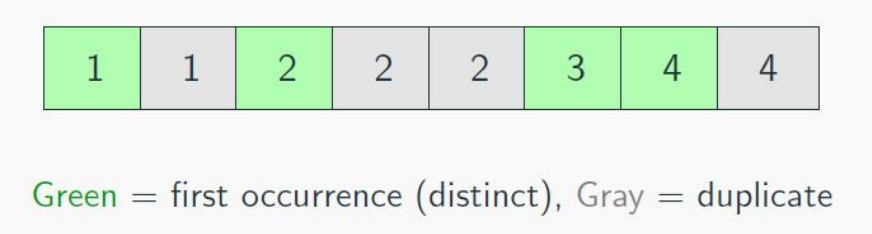
```
#include <stdio.h>

void leftRotate(int arr[], int n) {
      int first = arr [0]; // Save first element
      // Shift all elements left by 1
      for (int i = 0; i < n - 1; i++) {
            arr[i] = arr[i + 1];
      }
      arr[n - 1] = first; // Put first at end
}

int main() {
      int arr[] = {10 , 20, 30, 40, 50};
      int n = 5;
      leftRotate(arr , n);
      for (int i = 0; i < n; i++) printf ("%d ", arr[i]);
      // Output : 20 30 40 50 10
      return 0;
}
```

Distinct Elements in Sorted Array

Task: Count the number of distinct (unique) elements in a sorted array.



Key Insight:

In a sorted array, duplicates are always adjacent.
Count when current ≠ previous.

Algorithm:

```
count ← 1
for i ← 1 to n-1 do
if arr[i] ≠ arr[i-1] then
count ← count + 1
return count
```

Answer: 4 distinct elements (1, 2, 3, 4)

Distinct Elements: Code

```
#include <stdio.h>

int countDistinct(int arr[], int n) {
      if (n == 0) return 0;
      int count = 1; // First element is always distinct
      for (int i = 1; i < n; i++) {
            if (arr[i] != arr[i - 1]) {
                  count++;
            }
      }
      return count;
}

int main () {
      int arr[] = {1, 1, 2, 2, 2, 3, 4, 4};
      int n = 8;
      printf ("Distinct elements : %d\n", countDistinct(arr , n));
      // Output: 4
      return 0;
}
```

Distinct Elements in Unsorted Array

Task: Count distinct elements when array is not sorted.

Approach: For each element, check if it appeared before.

```
int countDistinctUnsorted(int arr[], int n) {
      int count = 0;
      for (int i = 0; i < n; i++) {
            // Check if arr[i] appeared before index i
            int isDuplicate = 0;
            for (int j = 0; j < i; j++) {
                  if (arr[j] == arr[i]) {
                        isDuplicate = 1;
                        break;
                  }
            }
            if (!isDuplicate) count++;
      }
      return count;
}
// arr = {5, 2, 8, 2, 5, 1} -> Distinct : 4 (5, 2, 8, 1)
```

Note: This is $O(n^2)$. For better performance, sort first then use the sorted algorithm.

Distinct Elements: O(n) Approach (Frequency Array)

Scenario: If we know the range of values is small (e.g., 0 to 100).

Idea: Use another array (frequency map) to track seen elements.

```
int countDistinctOptimized(int arr[], int n) {
     int freq[101] = {0}; // Assumes values are between 0 and 100
     int count = 0;

     for (int i = 0; i < n; i++) {
          if (freq[arr[i]] == 0) { // If seen for the first time
               count++;
               freq[arr[i]] = 1; // Mark as seen
          }
     }
     return count ;
}
```

Trade-off: Faster time $O(n)$, but uses extra space O(Range).

Searching Algorithms

Why Search Algorithms?

Common Problem: Given an array and a target value, does the value exist? Where?

Real-world Examples:

- Finding a contact in your phone
- Looking up a word in a dictionary
- Checking if an item is in stock
- Finding a student's record by ID

Two Main Approaches:

1. **Linear Search:** Check each element one by one
2. **Binary Search:** Divide and conquer (requires sorted array)

Problem: Find Maximum and Minimum

Idea: Start from the beginning, check each element until found or end reached.

```
int linearSearch(int arr[], int n, int target) {
      for (int i = 0; i < n; i++) {
            if (arr[i] == target ) {
                  return i; // Return index if found
            }
      }
      return -1; // Return -1 if not found
}
int main () {
      int arr[] = {64 , 34, 25, 12, 22, 11, 90};
      int n = 7;
      int target = 22;
      int result = linearSearch(arr, n, target);
      if (result != -1)
            printf ("Found at index %d\n", result); // Found at index 4
      else
            printf ("Not found \n");
      return 0;
}
```

Linear Search: Visualization

Searching for 22 in array:

Time Complexity: *O(n)* – may need to check all elements 23

The Guessing Game

Let's play a game:

- I am thinking of a number between **1** and **1000**.
- You guess a number.
- I will tell you if my number is **Higher**, **Lower**, or **Equal**.

Strategy 1 (Linear): Guess 1, 2, 3, 4...

- Worst case: 1000 guesses! (Too slow)

Strategy 2 (Binary): Guess 500 (Middle).

- If I say "Lower", the answer is 1-499. (We eliminated 500 numbers!)
- Next guess: 250.
- If I say "Higher", the answer is 251-499.
- Max guesses needed: ≈ 10 (2^{10} = 1024)
- This is the power of Binary Search!

This is the power of Binary Search!

Binary Search: Concept

Prerequisite: Array must be sorted!

Idea: Divide the search space in half at each step.

1. Compare target with middle element
2. If equal, found!
3. If target < middle, search left half
4. If target > middle, search right half
5. Repeat until found or search space is empty

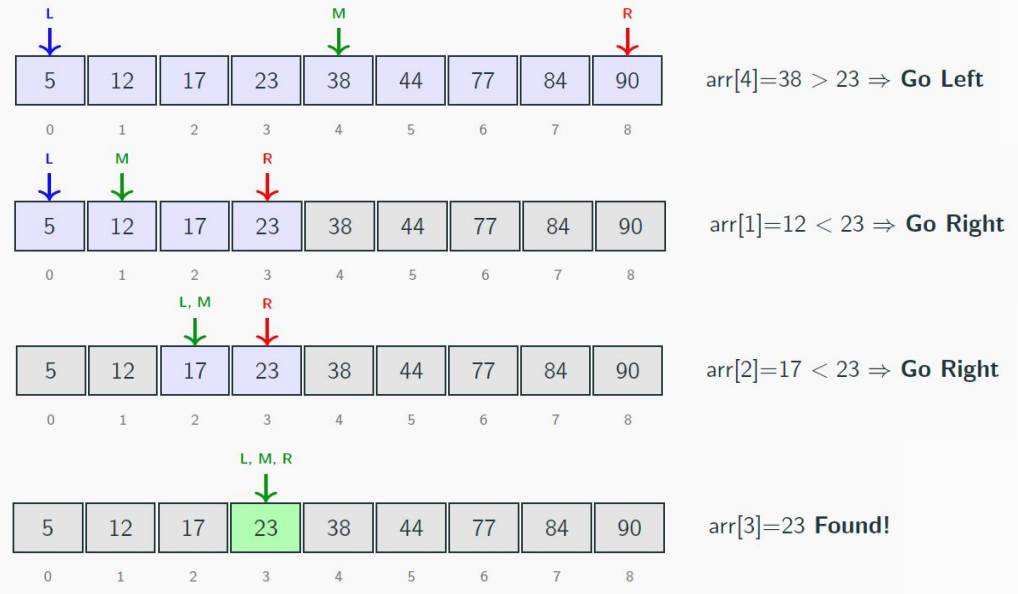
Analogy: Finding a word in a dictionary

- Open to the middle
- Is the word before or after?
- Go to the appropriate half and repeat

Time Complexity: O(log n) – much faster than linear search!

Binary Search: Step-by-Step

Target: 23 Array: [5, 12, 17, 23, 38, 44, 77, 84, 90]



Summary: Found 23 at index 3 in 4 comparisons.

Binary Search: Code

```
int binarySearch(int arr[], int n, int target) {
      int left = 0, right = n - 1;

      while ( left <= right ) {
            int mid = left + ( right - left ) / 2; // Avoid overflow

            if (arr[mid] == target) {
                  return mid; // Found!
            } else if (arr[mid] < target ) {
                  left = mid + 1; // Search right half
            } else {
                  right = mid - 1; // Search left half
            }
      }

      return -1; // Not found
}
// Usage:
int arr[] = {5, 12, 17, 23, 38, 44, 77, 84, 90};
int result = binarySearch(arr, 9, 23) ; // Returns 3
```

Search Algorithm Comparison

Header	Linear Search	Binary Search
Time Complexity	`O(n)`	`O(log n)`
Requires Sorted?	`No`	**`Yes`**
Best Case	`O(1)`	`O(1)`
Worst Case	`O(n)`	`O(log n)`
Space Complexity	`O(1)`	`O(1)`

When to use which?

- **Linear:** Small arrays, unsorted data, single search
- **Binary:** Large sorted arrays, frequent searches

Example: Searching in 1,000,000 elements

- **Linear:** up to 1,000,000 comparisons
- **Binary:** at most 20 comparisons! (log2 1000000 ≈ 20)

Bubble Sort

Why Sorting?

Sorting = arranging elements in a specific order (ascending/descending)

Why is sorting important?

- Enables Binary Search ($O(\log n)$ instead of $O(n)$)
- Easier to find duplicates, min, max
- Data presentation (reports, leaderboards)
- Foundation for many algorithms

Sorting Algorithms We'll Learn:

1. **Bubble Sort** – Simple, educational
2. **Selection Sort** – Find minimum, place it
3. **Insertion Sort** – Like sorting cards

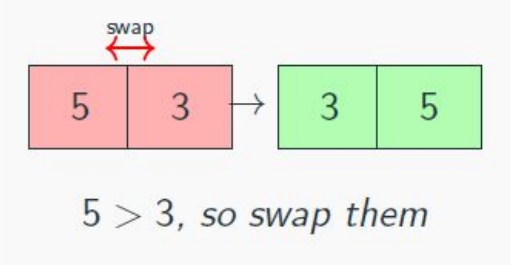
All three have $O(n^2)$ time complexity but differ in behavior.

Bubble Sort: Concept

Idea: Repeatedly compare adjacent elements and swap if out of order. Larger elements “bubble up” to the end.

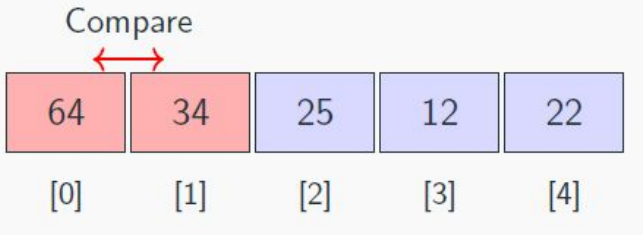
Algorithm:

1. Compare first two elements, swap if needed
2. Move to next pair, compare and swap
3. Continue until end of array (one pass)
4. Largest element is now at the end!
5. Repeat for remaining unsorted portion



Bubble Sort: Pass 1, Step 1

Array: [64, 34, 25, 12, 22] Target: Sort ascending



Compare `arr[0]` and `arr[1]`: 64 > 34? **Yes** ⇒ Swap!

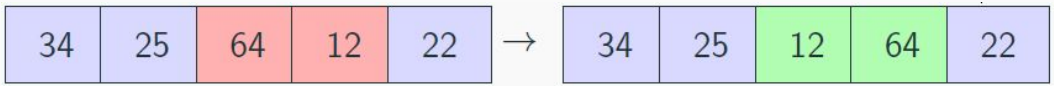
After swap:

Bubble Sort: Pass 1, Steps 2-3

Current: [34, 64, 25, 12, 22]

Step 2: Compare arr[1] and arr[2]: 64 > 25? **Yes** ⇒ Swap!

Step 3: Compare arr[2] and arr[3]: 64 > 12? **Yes** ⇒ Swap!



Bubble Sort: Pass 1, Step 4 – Complete

Current: [34, 25, 12, 64, 22]

Step 4: Compare arr[3] and arr[4]: 64 > 22? **Yes** ⇒ Swap!

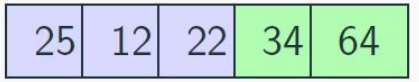
Pass 1 Complete!

- **64** (the largest) has “bubbled” to the end
- It’s now in its final sorted position
- Next pass only needs to sort indices 0-3

After Pass 1: [34, 25, 12, 22, 64]

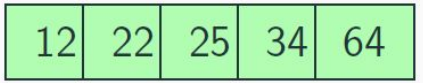
Bubble Sort: Remaining Passes

Pass 2: Sort [34, 25, 12, 22] → [25, 12, 22, 34]



Pass 3: Sort [25, 12, 22] → [12, 22, 25]

Pass 4: Sort [12, 22] → [12, 22] (no swaps needed!)



Sorted! [12, 22, 25, 34, 64]

Bubble Sort: Code

```
void bubbleSort(int arr[], int n) {
      for (int i = 0; i < n - 1; i++) { // n-1 passes
            for (int j = 0; j < n - 1 - i; j++) { // Compare adjacent
                  if (arr[j] > arr[j + 1]) {
                        // Swap arr[j] and arr[j +1]
                        int temp = arr[j];
                        arr[j] = arr[j + 1];
                        arr[j + 1] = temp ;
                  }
            }
      }
}

int main () {
      int arr[] = {64, 34, 25, 12, 22};
      bubbleSort(arr, 5);
      // arr is now: {12, 22, 25, 34, 64}
      return 0;
}
```

Bubble Sort: Optimized Version

Optimization: If no swaps occur in a pass, array is already sorted!

```
void bubbleSortOptimized(int arr[], int n) {
     for (int i = 0; i < n - 1; i++) {
          int swapped = 0; // Track if any swap happened

          for (int j = 0; j < n - 1 - i; j++) {
               if (arr[j] > arr[j + 1]) {
                    int temp = arr[j];
                    arr[j] = arr[j + 1];
                    arr[j + 1] = temp;
                    swapped = 1;
               }
          }

          if (!swapped) break; // Already sorted, exit early
     }
}
```

Best case: Already sorted ⇒ *O(n)* (just one pass)

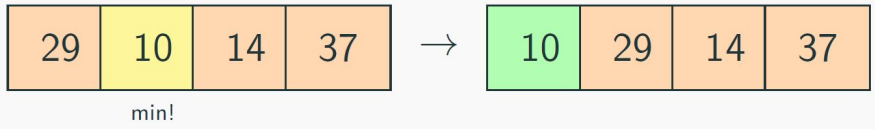
Selection Sort

Selection Sort: Concept

Idea: Repeatedly find the minimum element and place it in its correct position.

Algorithm:

1. Find the minimum element in the unsorted portion
2. Swap it with the first unsorted element
3. Move the boundary of sorted portion one step right
4. Repeat until entire array is sorted

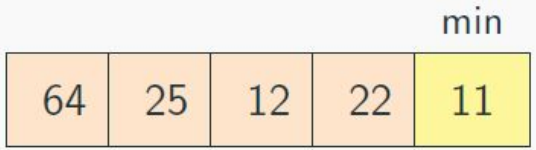


Find minimum (10), swap with first element

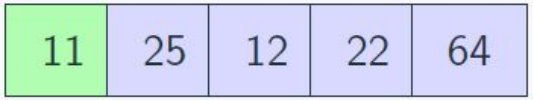
Selection Sort: Step 1

Array: [64, 25, 12, 22, 11]

Find minimum in entire array:



Swap min (11) with first (64):

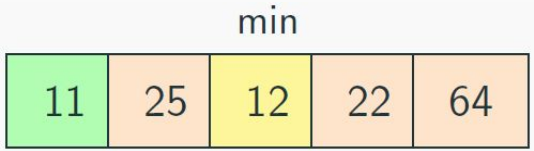


Result: 11 is now in its final position! → [11 , 25, 12, 22, 64]

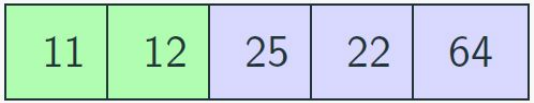
Selection Sort: Step 2

Array: [11 , 25, 12, 22, 64]

Find min in unsorted [1..4]:



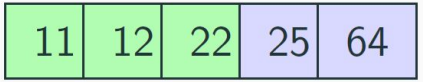
Swap 12 with first unsorted (25):



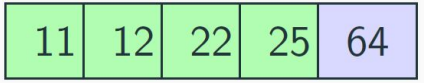
Result: [11, 12 , 25, 22, 64]

Selection Sort: Remaining Passes

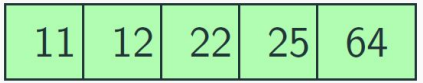
Step 3: Find min in [25, 22, 64] → 22. Swap with 25.



Step 4: Find min in [25, 64] → 25. Already in position!



Step 5: Only 64 left → Already in position!



Sorted! [11, 12, 22, 25, 64]

Selection Sort: Code

```
void selectionSort(int arr[], int n) {
      for (int i = 0; i < n - 1; i++) {
            // Find minimum in unsorted portion
            int minIdx = i;
            for (int j = i + 1; j < n; j++) {
                  if (arr[j] < arr[minIdx]) {
                        minIdx = j;
                  }
            }

            // Swap minimum with first unsorted element
            if (minIdx != i) {
                  int temp = arr[i];
                  arr[i] = arr[minIdx];
                  arr[minIdx] = temp;
            }
      }
}

// Usage : selectionSort(arr, 5);
```

Insertion Sort

Insertion Sort: Concept

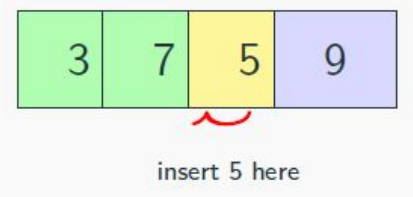
Idea: Build the sorted array one element at a time, inserting each new element into its correct position.

Analogy: Sorting playing cards in your hand

- Pick up one card at a time
- Insert it in the correct position among cards already in hand

Algorithm:

1. Consider first element as sorted
2. Take next element and insert into sorted portion
3. Shift larger elements to the right to make room
4. Repeat for all elements

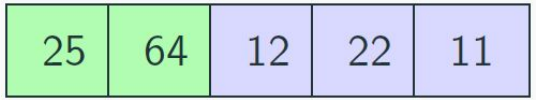


Insertion Sort: Step 1

Array: [64, 25, 12, 22, 11]

Starting state (64 is sorted):

Insert 25 (25 < 64, shift):

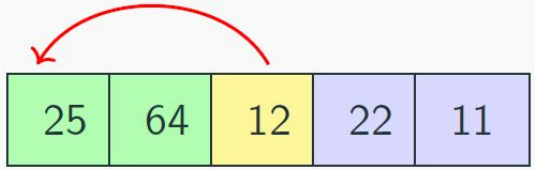


Result: [25, 64 , 12, 22, 11]

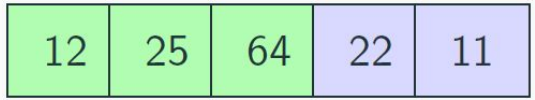
Insertion Sort: Step 2

Array: [25, 64 , 12, 22, 11]

Insert 12 (shift 64, shift 25):



After insertion:



Result: [12, 25, 64 , 22, 11]

Insertion Sort: Remaining Steps

Step 3: Insert 22 into [12, 25, 64] → goes between 12 and 25.

Step 4: Insert 11 into [12, 22, 25, 64] → smallest, goes to position 0.

Sorted! [11, 12, 22, 25, 64]

Insertion Sort: Code

```
void insertionSort(int arr[], int n) {
      for (int i = 1; i < n; i++) {
      int key = arr[i]; // Element to insert
      int j = i - 1;
      // Shift elements greater than key to the right
      while (j >= 0 && arr[j] > key ) {
            arr[j + 1] = arr[j];
            j--;
      }
      // Insert key at correct position
      arr[j + 1] = key;
      }
}

int main () {
      int arr[] = {64 , 25, 12, 22, 11};
      insertionSort(arr, 5);
      // arr is now: {11 , 12, 22, 25, 64}
      return 0;
}
```

Sorting Algorithm Comparison

Algorithm	Best	Average	Worst	Stable?
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	Yes
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	No
Insertion Sort	$O(1)$	$O(n^2)$	$O(n^2)$	Yes

When to use which?

- **Bubble Sort:** Educational purposes, rarely used in practice
- **Selection Sort:** When memory writes are expensive
- **Insertion Sort:** Nearly sorted data, small arrays, online sorting

Stable Sort: Maintains relative order of equal elements.

Note: For large arrays, use *O(n log n)* algorithms like Merge Sort or Quick Sort.

Summary

Key Takeaways

1. **Arrays** store multiple values of the same type in contiguous memory
2. Arrays are **zero-indexed**: first element is arr[0]
3. Always be careful of **array bounds** – C doesn't check!
4. Arrays are **passed by reference** to functions
5. **Linear Search:** Simple but slow ($O(n)$)
6. **Binary Search:** Fast ($O(\log n)$) but requires sorted array
7. **Bubble Sort:** Compare adjacent, bubble up largest
8. **Selection Sort:** Find minimum, place in position
9. **Insertion Sort:** Insert each element in sorted order

Golden Rules for Arrays

Do’s

- Always initialize arrays before use
- Pass array size as a parameter to functions
- Use loops for array operations
- Check bounds before accessing elements

Don’ts

- Don’t access indices outside [0, size-1]
- Don’t assume array size inside functions (use parameter)
- Don’t forget that arrays decay to pointers
- Don’t use sizeof on array parameters

Acknowledgement

- Mohammad Sadat Hossain, Lecturer, CSE, BUET